

SoundProof

Dossier de projet

Bloc 2 — Concevoir et développer des applications logicielles

Titre RNCP 39583 — Expert en Développement Logiciel

Application mobile de gestion de salles de répétition musicale

Le code source complet et les 11 documents détaillés (docs/01 à docs/11) sont fournis dans le dépôt Git accompagnant ce dossier.

Sommaire

1. Présentation du projet
2. Protocole de déploiement continu — C2.1.1
3. Critères de qualité et de performance — C2.1.1
4. Protocole d'intégration continue — C2.1.2
5. Architecture logicielle et maintenabilité — C2.2.1
6. Présentation du prototype — C2.2.1
7. Frameworks et paradigmes de développement — C2.2.1
8. Harnais de tests unitaires — C2.2.2
9. Mesures de sécurité — C2.2.3
10. Accessibilité — C2.2.3
11. Historique des versions — C2.2.4
12. Dernière version fonctionnelle — C2.2.4
13. Cahier de recettes — C2.3.1
14. Plan de correction des bogues — C2.3.2
15. Manuel de déploiement — C2.4.1
16. Manuel d'utilisation — C2.4.1
17. Manuel de mise à jour — C2.4.1
18. Correspondance compétences ↔ preuves

1. Présentation du projet

1.1 Contexte et besoin

SoundProof répond à un besoin concret des musiciens : réserver simplement une salle de répétition équipée, depuis leur téléphone, sans conflit de créneaux, et se coordonner avec les autres utilisateurs de la salle. Le choix d'une application mobile découle directement de cet usage nomade : on réserve une salle ou l'on consulte le chat entre deux déplacements, pas assis devant un poste de travail. Cet argument justifie l'équipement ciblé exigé par la compétence C2.2.1.

Deux rôles structurent l'application : **MEMBER** (consulte les salles, réserve créneaux et matériel, gère ses réservations, participe au chat des salles où il a réservé) et **ADMIN** (gère le parc de salles et de matériel, supervise toutes les réservations).

1.2 Périmètre fonctionnel — 10 user stories

#	User story	Critère d'acceptation clé
US1	Créer un compte avec un mot de passe fort	MDP ≥ 12 car. (min/maj/chiffre), email unique
US2	Se connecter et rester connecté	Session restaurée sans ressaisie, refresh sécurisé
US3	Consulter les salles et leur équipement	Liste des salles actives + équipements
US4	Voir les disponibilités sur une semaine	Grille 30 min, créneaux occupés non sélectionnables
US5	Réserver un créneau sans chevauchement	Conflit → 409 ; règles de validité appliquées
US6	Ajouter du matériel de la salle	Matériel indisponible refusé avec son nom
US7	Consulter et annuler ses réservations	Annulation confirmée ; passée non annulable
US8	Accéder au chat temps réel de la salle	Refus sans réservation ; réception < 500 ms
US9	(Admin) Gérer salles et matériel	Écritures ADMIN ; désactivation logique
US10	(Admin) Gérer toutes les réservations	Liste filtrable, annulation de toute réservation

1.3 Architecture générale

Un client mobile unique communique avec une API par deux canaux : REST (/api/v1) pour les opérations classiques, WebSocket pour le chat temps réel. La persistance est assurée par PostgreSQL via l'ORM Prisma.

Application mobile React Native / Expo TanStack Query Socket.IO client	→ REST /api/v1 → ↔ WebSocket ↔	API — NestJS REST + WebSocket Guards JWT · Helmet ValidationPipe	→ Prisma →	PostgreSQL 16 Base relationnelle
--	-----------------------------------	--	------------	--

1.4 Stack technique et structure du dépôt

Couche	Technologie
Application mobile	React Native / Expo (SDK 54, TypeScript) + Expo Router
Data-fetching / stockage	TanStack Query v5 + Axios ; jetons dans expo-secure-store
Backend	NestJS 11 (TypeScript)
Base / ORM	PostgreSQL 16 / Prisma 6
Temps réel	Socket.IO (@nestjs/websockets + socket.io-client)
Authentification	JWT (access + refresh), hachage argon2
Tests	Jest, Supertest, React Native Testing Library, Maestro
Qualité	ESLint, Prettier, Husky, lint-staged, commitlint
CI/CD & conteneurs	GitHub Actions, EAS Build, Docker + Docker Compose

Le dépôt est un **monorepo** : backend/ (API NestJS), mobile/ (application Expo), docs/ (11 documents détaillés), .github/ (workflows CI/CD, Dependabot, template d'issue), et les fichiers de conteneurisation.

1.5 Données de démonstration

Un jeu de données de démonstration (seed) permet de tester et de démontrer l'application immédiatement : 4 comptes (1 administrateur, 3 membres), 3 salles avec 4 à 6 équipements chacune, des réservations (passées, à venir, annulée) et des messages de chat. Mot de passe commun : SoundProof2026!.

Rôle	Compte	Particularité
ADMIN	admin@soundproof.fr	accès à l'onglet Administration
MEMBER	marie@soundproof.fr	réservation passée + futures sur le Studio A (accès chat)
MEMBER	karim@soundproof.fr	réservation future Studio A, une réservation annulée
MEMBER	lea@soundproof.fr	réservation future Studio B

► Détail : [README.md](#), [docs/04-architecture-logicielle.md](#), [backend/prisma/seed.ts](#).

2. Protocole de déploiement continu — C2.1.1

Le protocole couvre deux artefacts distincts : l'API (conteneur Docker) et l'application mobile (binaire Android/iOS produit par EAS Build). Son implémentation est le workflow `.github/workflows/cd.yml`.

2.1 Environnements

Environnement	API	Mobile	Base de données	Usage
development	Locale (watch)	Expo Go	PostgreSQL Docker	Développement

Environnement	API	Mobile	Base de données	Usage
test	Démarrée par la CI	Bundle vérifié	PostgreSQL éphémère	Validation auto.
preview	Conteneurs de démo	APK installable	PostgreSQL conteneur	Recette / démo
production	compose.prod.yml	AAB (Play Store)	PostgreSQL non exposée	Cible finale

2.2 Séquence de déploiement de l'API

Le push d'un tag sur main déclenche le pipeline `cd.yml`, qui automatise la production et la publication des artefacts (quatre premières étapes). Le déploiement sur l'hôte et sa vérification sont la procédure d'exploitation documentée** (`docs/09`, `docs/01`), non incluse dans le pipeline :

- Exécution complète de la CI (réutilisation via `workflow_call`) — aucun déploiement si la CI échoue.
- Build de l'image Docker multi-stage (build TypeScript → runtime minimal non-root).
- Publication sur GitHub Container Registry (`ghcr.io`), taguée version et latest.
- Création automatique d'une GitHub Release reprenant la section du `CHANGELOG.md`.
- Déploiement sur l'hôte (procédure d'exploitation, hors pipeline) : `docker compose -f docker-compose.prod.yml up -d` récupère l'image publiée ; les migrations Prisma s'appliquent automatiquement au démarrage du conteneur (entrypoint), avant que l'API n'accepte des requêtes.
- Vérification : healthcheck `GET /api/v1/health` (statut app + BDD) + smoke test documenté.

2.3 Séquence de déploiement du mobile

- Même déclencheur (tag `v*.*.*`).
- Build EAS : `eas build --platform android --profile preview`, authentifié par le secret `EXPO_TOKEN`.
- Récupération de l'APK et attachement à la GitHub Release (téléchargeable et installable).
- Distribution : lien de la Release + QR code EAS. Publication sur les stores documentée comme étape ultérieure.

2.4 Secrets et prérequis

Élément	Où	Usage
EXPO_TOKEN	Secret GitHub Actions	Authentification de eas build en CI (compte Expo gratuit)
GITHUB_TOKEN	Fourni par GitHub Actions	Push de l'image sur <code>ghcr.io</code> , création de la Release

Élément	Où	Usage
.env de production	Machine cible uniquement	Secrets JWT, mot de passe PostgreSQL (jamais committé)

2.5 Déploiement progressif et rollback

Chaque fusion dans develop est immédiatement testable dans Expo Go ; chaque tag produit un APK versionné installable. Rollback : redéploiement de l'image taguée précédente (API_VERSION=vX.Y.Z + up -d) et réinstallation de l'APK de la Release antérieure, chaque Release conservant son binaire. Ce protocole a été réellement exécuté lors de la publication de la v1.0.0 : image, Release et APK ont été produits automatiquement.

► *Détail : docs/01-protocole-deploiement-continu.md.*

3. Critères de qualité et de performance — C2.1.1

Chaque critère est mesurable, associé à un outil de mesure et à un moment de vérification précis.

Catégorie	Critère	Cible	Outil	Vérifié
Code	Erreurs de lint	0 erreur / 0 warning	ESLint	pre-commit + CI
Code	Typage strict	strict:true, tsc OK	TypeScript	CI
Code	Commits conventionnels	100 % conformes	commitlint	commit-msg
Mobile	Projet Expo sain	18/18 checks	expo-doctor	CI
Tests	Couverture logique métier	≥ 80 % lignes/branches	Jest	CI (bloquant)
Tests	e2e API critiques	100 % passants	Supertest	CI
Perf.	Réponse API (p95 CRUD)	< 200 ms local	logs / test	recette
Perf.	Démarrage à froid app	< 3 s	chronométrage	recette
Perf.	Navigation entre écrans	< 300 ms, sans gel	observation	recette
Perf.	Latence message chat	< 500 ms local	test chronométré	recette
Access.	WCAG 2.1 AA	0 anomalie bloquante	TalkBack + Scanner	recette
Sécu.	Vulnérabilités deps	0 haute / critique	npm audit	CI (main)

Environnement de développement détaillé

- **Éditeur** : VS Code + extensions ESLint, Prettier, Prisma, Expo Tools. **Runtime** : Node.js 20 LTS, npm (lockfiles committés).
- **Compilateurs** : tsc côté backend (piloté par la CLI NestJS) ; **Metro** (bundler) + moteur Hermes côté mobile ; vérification des types par tsc --noEmit.

- **Serveur d'application** : NestJS/Express en mode watch (dev), Node.js dans un conteneur Docker (prod) ; serveur de développement Expo pour l'application.
- **Exécution mobile** : Expo Go (dev), build natif EAS (prod) ; émulateur Android (Android Studio) et/ou appareil physique.
- **Gestion des sources** : git + GitHub. Base : PostgreSQL 16, gérée par Prisma (migrations versionnées + Prisma Studio).

► *Détail : docs/02-criteres-qualite-performance.md.*

4. Protocole d'intégration continue — C2.1.2

Objectif : fusionner régulièrement les codes sources dans une branche partagée (develop) et détecter les régressions au plus tôt. **Déclencheurs** : push sur toute branche + pull request vers develop/main ; réutilisable par le déploiement continu (workflow_call).

4.1 Séquence du pipeline (ci.yml)

Le pipeline est organisé en jobs parallèles ; l'échec d'une étape fait échouer le pipeline :

Job	Étapes
backend	npm ci → lint → typecheck → tests unitaires avec couverture (< 80 % = échec) → build
backend-e2e	PostgreSQL de service → npm ci → migrations Prisma → tests Supertest
mobile	npm ci → lint → typecheck → tests avec couverture → expo-doctor → export du bundle Android
audit	npm audit --audit-level=high (non bloquant en dev, bloquant sur main et les tags)

Les seuils de couverture sont bloquants (config Jest des deux applications) : la CI échoue sous 80 %. Le résumé de couverture est publié dans le job summary de GitHub Actions.

4.2 Règles de fusion et défense locale

Toute intégration dans develop/main passe par une pull request avec CI verte requise et auto-revue documentée (projet solo). En amont, deux hooks git Husky constituent la première ligne de défense :

Hook	Outil	Action
pre-commit	lint-staged	ESLint --fix + Prettier sur les fichiers stagés uniquement
commit-msg	commitlint	Rejet des messages non conformes aux Conventional Commits

Ce dispositif a réellement intercepté plusieurs anomalies avant livraison (voir §14), dont une vulnérabilité de dépendance détectée au moment du tag v1.0.0 — preuve concrète de son efficacité.

5. Architecture logicielle et maintenabilité — C2.2.1

5.1 Backend — architecture modulaire NestJS

Le backend suit une architecture modulaire (un module par domaine métier) et le pattern Controller → Service → Repository (Prisma). Chaque contrôleur ne fait que valider (DTO class-validator) et déléguer ; le service porte la logique ; l'accès aux données passe exclusivement par Prisma. Cette séparation stricte des responsabilités = maintenabilité : chaque couche se teste et évolue isolément, et l'injection de dépendances native permet de substituer les dépendances par des mocks en test.

Module	Responsabilité
AuthModule	register/login/refresh/logout ; stratégie JWT ; guards JwtAuthGuard, RolesGuard + @Roles()
UsersModule	profil courant (GET/PATCH /users/me)
RoomsModule	CRUD salles (écriture ADMIN), liste/détail, disponibilités
EquipmentModule	CRUD matériel par salle (écriture ADMIN)
BookingsModule	création/annulation, règles anti-chevauchement en transaction
ChatModule	gateway Socket.IO + historique REST paginé
HealthModule	GET /api/v1/health (statut app + BDD) via Terminus

5.2 Règles métier critiques (BookingsService)

La logique de réservation concentre les règles les plus sensibles, testées en priorité (§8) :

#	Règle
1	startsAt < endsAt, créneau dans le futur, durée entre 30 min et 8 h, alignement sur des pas de 30 min
2	Anti-chevauchement salle : refus 409 si un booking CONFIRMED chevauche, contrôle dans une transaction Prisma
3	Anti-chevauchement matériel : même règle par équipement ; l'équipement doit appartenir à la salle réservée
4	Annulation : par le propriétaire ou un ADMIN, uniquement si le créneau est futur ; statut → CANCELLED (pas de suppression)
5	Accès au chat : au moins un booking CONFIRMED (passé ou futur) sur la salle, ou rôle ADMIN

5.3 Contrat d'API REST (préfixe /api/v1)

Méthode	Route	Accès	Description
POST	/auth/register · /login · /refresh · /logout	public / auth	Cycle d'authentification
GET/PATCH	/users/me	authentifié	Profil courant
GET	/rooms · /rooms/:id	authentifié	Liste et détail des salles
GET	/rooms/:id/availability	authentifié	Créneaux occupés sur une période
POST/PATCH/DELETE	/rooms · /rooms/:id/equipment	ADMIN	Gestion salles et matériel
POST	/bookings	authentifié	Créer une réservation
GET	/bookings/me · /bookings	auth / ADMIN	Mes réservations / toutes
DELETE	/bookings/:id	propriétaire/ADMIN	Annulation
GET	/rooms/:id/messages	membre de la salle	Historique paginé du chat

L'API est documentée par **Swagger** (/api/docs, désactivé en production).

5.4 Modèle de données

User id, email, passwordHash, role, refreshTokenHash	Room id, name, description, capacity, isActive	Equipment id, name, category, roomId, isActive
Booking id, userId, roomId, startsAt, endsAt, status	BookingEquipment bookingId, equipmentId (table de jointure)	Message id, content, userId, roomId, createdAt

Relations : User 1–N Booking · Room 1–N Booking/Equipment/Message · Booking N–N Equipment (via BookingEquipment) · User 1–N Message.

- **Annulation logique** (status=CANCELLED, isActive=false) : aucune suppression, l'historique est conservé (traçabilité).
- **BookingEquipment** : le matériel n'est reservable que dans le cadre d'une réservation de salle (cascade avec la réservation).
- **Index composites** (roomId, startsAt, endsAt) et (roomId, createdAt) : optimisent la recherche de chevauchement et la pagination du chat.

5.5 Temps réel — événements Socket.IO (namespace /chat)

Sens	Événement	Règle
client → serveur	room:join	vérifie l'accès à la salle, rejoint la room
client → serveur	message:send	valide (1-1000 car.), persiste, diffuse
serveur → clients	message:new	diffusé à la room avec l'auteur
serveur → client	error	accès refusé / validation

L'authentification WebSocket se fait par le JWT passé au handshake, vérifié dans le gateway. Côté mobile, la reconnexion est gérée explicitement : re-join automatique de la room et rafraîchissement de l'historique au retour au premier plan.

5.6 Organisation du code mobile

L'application sépare les écrans (routes) de la logique, ce qui facilite la maintenance et les tests :

Dossier	Rôle
app/	écrans = routes (Expo Router) : login, register, (tabs), rooms/[id], chat
src/api/	client HTTP (Axios) et appels API par domaine
src/components/	composants d'interface réutilisables (Button, Input, Modal, Toast...)
src/features/	logique par fonctionnalité (auth, chat)
src/hooks/	hooks partagés (useAuth, useNow...)
src/lib/	fonctions pures (calcul de créneaux, thème)

► Détail : docs/04-architecture-logicielle.md, backend/prisma/schema.prisma, mobile/src/.

6. Présentation du prototype — C2.2.1

Le prototype a d'abord été maqueté (wireframes basse-fidélité des 6 écrans, docs/04 §8) puis implémenté à l'identique. Il couvre les 6 écrans principaux, conçus selon les spécificités ergonomiques mobiles, avec une navigation par barre d'onglets en bas (Salles / Réservations / Admin si ADMIN / Profil).

Écran (route Expo Router)	Contenu clé
/login, /register	formulaires validés (react-hook-form + zod), erreurs par champ, clavier géré (KeyboardAvoidingView)
/(tabs)/rooms	cartes salle (nom, capacité, équipements), pull-to-refresh, états vide/chargement/erreur
/rooms/[id]	grille hebdomadaire (pas 30 min), créneaux occupés grisés, sélection début/fin, cases matériel
/(tabs)/bookings	onglets À venir / Passées, annulation avec dialogue de confirmation natif
/rooms/[id]/chat	liste inversée à défilement infini, indicateur de connexion, envoi temps réel
/(tabs)/admin	CRUD salles/matériel, supervision des réservations (onglet masqué pour les membres)

Choix ergonomiques justifiés : barre d'onglets en bas (zone du pouce) ; zones tactiles $\geq 44 \times 44$ pt ; gestes standards (pull-to-refresh) ; **feedback systématique (toasts de succès/erreur après chaque action)** ; états vide/chargement/erreur distincts ; gestion du clavier virtuel (le champ actif reste

visible) ; SafeAreaView partout (encoches) ; orientation portrait verrouillée (flux verticaux — choix assumé et documenté).

6.1 Écran de réservation — description

L'écran de détail d'une salle illustre le soin ergonomique : la description et la capacité en tête, puis un navigateur de semaine (◀ ▶), une bande de jours sélectionnables, et la grille des créneaux par pas de 30 minutes. Les créneaux occupés sont grisés et barrés (distinction non uniquement colorimétrique — critère WCAG 1.4.1). La sélection d'un début fait apparaître les fins possibles (bornées par le créneau réservé suivant), puis les cases de matériel de la salle, un récapitulatif et le bouton de confirmation. Toute action asynchrone affiche un état de chargement et un toast de résultat.

6.2 Composants d'interface réutilisables

L'interface repose sur des composants réutilisables (src/components/) : Button, Input, Select, Modal, Toast, Spinner, EmptyState, Card. Le style provient d'un thème centralisé (src/lib/theme.ts : couleurs, espacements, tailles) — aucune valeur en dur dans les écrans, ce qui facilite la cohérence et l'évolution (mode sombre, contrastes).

Exigences de sécurité au niveau du prototype : les zones réservées (onglet admin, routes protégées) sont matérialisées côté interface, mais le contrôle réel est systématiquement côté API (un APK se décompile). L'application est fonctionnelle et manipulable en autonomie, testée sur appareil réel (voir §12).

► Détail : docs/04-architecture-logicielle.md §8-9, mobile/app/, mobile/src/components/.

7. Frameworks et paradigmes de développement — C2.2.1

7.1 Frameworks et justification

Choix	Justification
React Native / Expo	Un seul code TypeScript Android+iOS ; outillage Expo décisif en solo (Expo Go, builds cloud EAS sans Android Studio configuré, OTA possibles)
Expo Router	Navigation par fichiers : routes lisibles, zones protégées matérialisées, typage des routes
NestJS	Architecture modulaire, injection de dépendances native (testabilité), écosystème intégré (Passport/JWT, ValidationPipe, WebSockets, Terminus, Swagger)
Prisma	Schéma unique versionné, migrations reproductibles, client entièrement typé (base → écran), requêtes paramétrées
Socket.IO	Rooms natives adaptées au chat par salle, reconnexion automatique — précieuse sur réseau mobile

7.2 Paradigmes

- **Backend — orienté objet + injection de dépendances** : classes, décorateurs, encapsulation par module ; inversion de contrôle (les services reçoivent leurs dépendances par le constructeur, d'où la testabilité).
- **Mobile — programmation fonctionnelle** : composants fonction purs, composition, hooks (état et effets isolés), immutabilité ; logique métier extraite en fonctions pures (src/lib/slots.ts) testables exhaustivement.
- **Transverse — typage statique TypeScript strict** des deux côtés (strict: true) : les contrats d'API (DTO) et les modèles Prisma portent les types de bout en bout.

► Détail : docs/04-architecture-logicielle.md §2.

8. Harnais de tests unitaires — C2.2.2

Le harnais prévient les régressions via des seuils de couverture bloquants en CI (coverageThreshold Jest : la CI échoue sous 80 %).

8.1 Bilan mesuré

Suite	Nombre	Portée
Unitaires backend (Jest, Prisma mocké)	80	services auth, bookings, rooms, equipment, users, chat + gateway
e2e API (Supertest + PostgreSQL réelle)	20	parcours critiques de bout en bout
Unitaires mobile (Jest + RNTL)	51	créneaux, schémas zod, hook d'auth, écrans login/réservation, bulle de chat

Service backend	Lignes	Branches
auth.service	100 %	89 %
bookings.service	100 %	98 %
rooms.service	100 %	91 %
chat.service	100 %	94 %
users.service	100 %	83 %
equipment.service	94 %	92 %

8.2 Fonctionnalité couverte en priorité : la réservation

Le service BookingsService concentre la logique métier critique. Anti-chevauchement, vérification + insertion dans une transaction Prisma (anti-condition de course) :

```

return this.prisma.$transaction(async (tx) => {
  const conflict = await tx.booking.findFirst({
    where: { roomId, status: BookingStatus.CONFIRMED,
      startsAt: { lt: newEnd }, endsAt: { gt: newStart } },
  });
  if (conflict) throw new ConflictException('Ce créneau est déjà réservé pour cette salle.');
```

Cas testés : créneau invalide (début $\geq$ fin, passé, durée hors 30 min–8 h, non aligné sur 30 min) ; chevauchement exact / partiel / englobant / englobé ; chevauchement avec un booking annulé (doit passer) ; matériel d’une autre salle ; matériel déjà pris ; annulation par autrui (403) ; annulation d’une réservation passée (400).

8.3 Tests unitaires mobile

Côté mobile, les tests couvrent la logique métier et les composants clés :

Cible	Ce qui est vérifié
Calcul des créneaux (fonction pure)	chevauchements (bornes exclusives), fins possibles plafonnées à 8 h, semaine lun→dim, alignement 30 min
Schémas de validation zod	politique de mot de passe (longueur, casse, chiffre), confirmation, email
Hook d’authentification	restauration de session (US2), login, logout, échec de refresh (mocks API + secure-store)
Écran de connexion	erreurs de validation sans appel API, message d’erreur API, soumission + redirection
Écran de réservation	créneaux occupés grisés/désactivés, parcours début→fin→matériel→confirmation
Bulle de message (chat)	rendu auteur/heure/contenu, labels d’accessibilité présents

Test de non-régression systématique : chaque correction de bogue (§14) est accompagnée d’un test automatisé qui échoue avant le correctif et passe après — garantissant que l’anomalie ne réapparaîtra pas.

8.4 Tests e2e des parcours critiques

Exécutés contre une base PostgreSQL réelle, ils déroulent : inscription → mot de passe faible refusé (400) → email dupliqué (409) → connexion → réservation salle+matériel (201) → conflit 409 → matériel d’une autre salle (400) → disponibilités → annulation par autrui refusée (403) → route admin (403 membre / 200 admin) → annulation par le propriétaire (204) → créneau redevenu reservable → accès chat (200 avec réservation / 403 sans) → cycle refresh/logout avec refresh révoqué (401).

► Détail : `backend/src/**/*.spec.ts`, `backend/test/parcours-critiques.e2e-spec.ts`, `mobile/src/**/*.__tests__/_`.

9. Mesures de sécurité — C2.2.3

Analyse structurée sur l'OWASP Top 10 (2021). Principe directeur : l'APK étant décompilable (le code JS, l'URL de l'API et le projectId sont lisibles), tous les contrôles de sécurité sont côté API ; ce que fait l'application n'est que du confort.

Faible	Risque dans SoundProof	Mesures
A01 Access Control	Annuler la réservation d'autrui, lire un chat interdit, accéder à l'admin	Guards JWT + rôles globaux (tout protégé par défaut) ; contrôle de propriété ; accès chat vérifié en REST + WS
A02 Cryptographic	Vol de mots de passe, détournement de session	argon2 (mots de passe + refresh hachés en BDD) ; secure-store (Keystore chiffré) ; secrets en variables
A03 Injection	Injection SQL, contenu malveillant dans le chat	Prisma (paramétré) ; ValidationPipe (whitelist + forbidNonWhitelisted) ; validation WebSocket ; texte inerte
A04 Insecure Design	Double réservation (course), perte de traçabilité	Règles métier en transaction ; annulation logique ; rate limiting
A05 Misconfiguration	Swagger public, BDD exposée, secret dans l'APK	Helmet ; Swagger off en prod ; conteneur non-root ; BDD sur réseau internal
A06 Vulnerable Comp.	CVE dans une dépendance	npm audit bloquant (main/tags) + Dependabot + expo-doctor
A07 Auth Failures	Brute force, énumération de comptes	MDP ≥ 12 car. ; throttling 5 req/min sur login ; messages génériques ; refresh révoqué au logout
A08 Integrity	Dépendance compromise, APK falsifié	Lockfiles + npm ci ; images versionnées ; builds EAS signés
A09 Logging	Incidents indétectables	Échecs d'auth et refus d'accès journalisés sans donnée sensible
A10 SSRF	(théorique)	Aucune requête sortante construite depuis une entrée utilisateur

Tests de sécurité exécutés — 8/8 OK

Scénario	Attendu	Obtenu
Accès API sans token	401	401
Route admin avec compte membre	403	403
Annulation de la réservation d'autrui	403	403
Injection SQL dans un champ texte	neutralisée	stockée comme texte, base intacte
XSS dans un message de chat	inerte	diffusé et stocké comme texte brut
Brute force sur le login	429	429
Chat d'une salle sans réservation	403	403
Élévation de rôle (payload non whitelisted)	400	400

Limites résiduelles assumées

- Le trafic de développement (Expo Go ↔ API locale) est en HTTP, acceptable sur réseau privé ; HTTPS documenté et exigé en production (terminaison TLS par reverse proxy).
- Des vulnérabilités moderate subsistent dans des dépendances transitives de l’outillage de build Expo, non embarquées dans l’APK ; suivies à chaque montée de SDK (sous le seuil bloquant high).
- Pas d’agrégation centralisée des logs ni d’alerting (hors périmètre d’un déploiement mono-machine) ; les journaux sont accessibles via docker logs.

► Détail : docs/05-securite-owasp.md, docs/07-cahier-de-recettes.md §4.

10. Accessibilité — C2.2.3

10.1 Référentiel choisi et justifié

WCAG 2.1 niveau AA, via la norme européenne EN 301 549. Le RGAA 4, souvent cité en France, est la déclinaison française des WCAG pour le web : sa méthode repose sur l’inspection du DOM HTML et ne couvre pas les applications mobiles natives. Pour le mobile, la référence légale française/européenne est l’EN 301 549 (chapitre 11 « logiciels »), qui applique les critères WCAG 2.1 AA aux applications mobiles. OPQUAST est retenu en complément méthodologique.

10.2 Mesures par principe

Principe	Mesures
Perceptible	accessibilityLabel sur tout élément interactif ; emojis décoratifs masqués ; contrastes tous $\geq 4.5:1$; police dynamique respectée (allowFontScaling jamais désactivé)
Utilisable	zones tactiles $\geq 44 \times 44$ pt ; composants tactiles natifs (Pressable) ; ordre de lecture = ordre visuel
Compréhensible	erreurs de formulaire par champ, annoncées (accessibilityLiveRegion) ; instructions avant saisie ; confirmations natives des actions destructives
Robuste	trio nom/rôle/état (accessibilityRole, accessibilityState) ; annonces dynamiques des toasts et des nouveaux messages de chat

10.3 Contrastes du thème mesurés

Couple (texte / fond)	Ratio	AA
Texte principal / blanc	17.74:1	OK
Texte secondaire / blanc	7.56:1	OK
Primaire (bleu) / blanc	6.70:1	OK
Danger / blanc	6.47:1	OK

Couple (texte / fond)	Ratio	AA
Succès / blanc	5.02:1	OK
Primaire / sélection (bleu clair)	5.49:1	OK

Les 12 couples du thème ont été validés par calcul (de 5.02:1 à 17.74:1).

10.4 Critères WCAG 2.1 AA couverts (extrait)

Critère	Mesure prise
1.1.1 Contenu non textuel	accessibilityLabel sur tout élément interactif ; emojis décoratifs masqués
1.3.1 Information et relations	rôles sémantiques (header, button, tab, checkbox, radio, progressbar)
1.4.3 Contraste minimum	tous les couples du thème $\geq 4.5:1$ (mesurés)
1.4.4 Redimensionnement du texte	allowFontScaling jamais désactivé ; testé à 200 %
2.5.5 Taille de la cible	zones tactiles $\geq 44 \times 44$ pt (constante MIN_TOUCH_SIZE)
3.3.1 Identification des erreurs	erreurs par champ, en toutes lettres, annoncées
4.1.2 Nom, rôle, valeur	accessibilityRole + accessibilityState (disabled, selected, checked, busy)
4.1.3 Messages d'état	toasts et nouveaux messages de chat annoncés (announceForAccessibility)

10.5 Protocole d'audit

L'audit est réalisé sur appareil réel selon un protocole précis, consigné dans docs/06 §3 :

Outil	Objet du contrôle
TalkBack (lecteur d'écran Android)	parcours complet des 6 écrans par balayage ; noms, rôles, états et annonces dynamiques lus correctement
Accessibility Scanner (Google)	analyse automatique : contrastes, tailles tactiles, libellés manquants
Taille de police système à 200 %	lisibilité, absence de texte tronqué bloquant
Calcul des contrastes du thème	ratios $\geq 4.5:1$ (réalisé, voir §10.3)

Limites connues : audit VoiceOver (iOS) non réalisé faute de matériel Apple ; les props React Native utilisées sont communes aux deux plateformes.

► *Détail : docs/06-accessibilite.md.*

11. Historique des versions — C2.2.4

Le projet suit le versionnage sémantique (SemVer). Chaque version correspond à un tag git réel et à une entrée du CHANGELOG.md (format Keep a Changelog). Le projet ayant été construit par couches, l'historique en rend compte fidèlement :

Version	Contenu	Date
v0.1.0	Socle technique, environnement de dev, CI/CD	2026-07-11

Version	Contenu	Date
v0.2.0	Conception, architecture, modèle de données, seed	2026-07-12
v0.3.0	API complète (auth, salles, matériel, réservations, chat)	2026-07-12
v0.4.0	Application mobile complète	2026-07-13
v0.5.0	Sécurité OWASP, accessibilité, harnais de tests renforcé	2026-07-13
v1.0.0	Recette + documentation d'exploitation complète	2026-07-14

Convention de commits : Conventional Commits (feat, fix, docs, test, chore, ci...), refusés à la source par commitlint. Stratégie de branches (Git Flow simplifié) : main (versions stables taguées), develop (intégration), branches de travail feat|fix|docs|chore/* fusionnées par merge commit.

► Détail : [CHANGELOG.md](#), [historique](#) et [tags git](#).

12. Dernière version fonctionnelle — C2.2.4

La v1.0.0 est fonctionnelle, fiable et viable. Sa publication a réellement exercé le pipeline complet : image Docker publiée sur GitHub Container Registry, GitHub Release générée depuis le CHANGELOG, et APK Android produit par EAS et attaché à la Release.

Preuves de viabilité :

- Les 10 user stories fonctionnent (application testée sur appareil réel via Expo Go ; parcours API vérifiés automatiquement par les tests e2e).
- CI verte sur main ; 80 + 20 + 51 tests passants ; expo-doctor 18/18 ; 0 vulnérabilité haute.
- L'APK constitue le livrable installable et manipulable en autonomie par un utilisateur, conformément au critère C2.2.4.

Recette et maintenance : la recette fonctionnelle a ensuite été exécutée sur appareil réel (24/24 conforme, §13). Elle a mis en évidence un unique défaut d'accessibilité (BUG-011), corrigé puis re-testé ; ce correctif de maintenance sera publié dans la prochaine version (v1.0.1), la v1.0.0 restant la dernière version taguée à la date du dossier.

► Détail : [GitHub Release v1.0.0 \(APK attaché\)](#) ; [image ghcr.io/gagawan/soundproof-api:v1.0.0](#).

13. Cahier de recettes — C2.3.1

Le cahier reprend l'ensemble des fonctionnalités et distingue trois familles de tests : fonctionnels, structurels et de sécurité. Préambule : environnement de recette (APK preview, base seedée, comptes de test), stratégie, convention de statut (OK / KO / N-T).

13.1 Tests fonctionnels — exemples de scénarios

Champ	Contenu (TF-001 — Inscription réussie)
User story	US1
Préconditions	Aucune session ; email non utilisé
Étapes	1. Écran Connexion → « S'inscrire ». 2. Saisir prénom, nom, email, mot de passe (×2). 3. Valider
Résultat attendu	Compte créé, utilisateur connecté, arrivée sur la liste des salles
Statut	OK — exécuté sur appareil réel

Champ	Contenu (TF-011 — Créneau en conflit)
User story	US5
Préconditions	Membre connecté ; Studio A avec une réservation confirmée sur le créneau
Étapes	1. Ouvrir Studio A. 2. Sélectionner un créneau chevauchant. 3. Confirmer
Résultat attendu	Message « Ce créneau est déjà réservé », aucune réservation créée
Statut	OK — exécuté sur appareil réel

Champ	Contenu (TF-015 — Annulation)
User story	US7
Préconditions	Membre avec une réservation à venir
Étapes	1. Onglet Réservations → À venir. 2. « Annuler ». 3. Confirmer dans le dialogue natif
Résultat attendu	Toast de confirmation ; réservation « Annulée » ; créneau redevenu libre
Statut	OK — exécuté sur appareil réel

Champ	Contenu (TF-016 — Accès au chat)
User story	US8
Préconditions	Membre ayant une réservation sur le Studio A
Étapes	1. Ouvrir Studio A → « Chat de la salle ». 2. Saisir un message. 3. Envoyer
Résultat attendu	Historique affiché, message visible immédiatement, indicateur « En ligne »
Statut	OK — après correction de BUG-011 (re-testé)

Champ	Contenu (TF-022 — Comportement hors connexion)
User story	— (spécifique mobile)
Préconditions	Application connectée
Étapes	1. Activer le mode avion. 2. Rafraîchir la liste des salles / tenter une réservation
Résultat attendu	Message d'erreur exploitable, aucun crash ; reprise normale au retour du réseau
Statut	OK — exécuté sur appareil réel

Le cahier compte 24 scénarios (TF) couvrant les 10 user stories en cas nominal et d'erreur. Aperçu de la couverture :

ID	US	Scénario	Type
TF-001 à 003	US1	Inscription (réussie, mot de passe faible, email dupliqué)	nominal + erreurs
TF-004 à 007	US2	Connexion, refus, session persistante, déconnexion	nominal + erreur
TF-008 / 009	US3/US4	Liste des salles, disponibilités hebdomadaires	nominal
TF-010 / 011	US5	Réservation libre / créneau en conflit	nominal + erreur
TF-012 / 013	US6	Réservation avec matériel / matériel indisponible	nominal + erreur
TF-014 / 015	US7	Consultation / annulation d'une réservation	nominal
TF-016 à 018	US8	Accès chat, temps réel entre deux comptes, refus sans réservation	nominal + erreur
TF-019 à 021	US9/US10	Administration salles/matériel/réservations, onglet masqué	nominal
TF-022 à 024	—	Hors connexion, reprise arrière-plan, clavier/rotation	spécifiques mobile

Campagne exécutée sur appareil réel : 24/24 conformes. La première passe a révélé un défaut d'accessibilité — un bouton d'action situé en bas d'écran (envoi de message dans le chat ; « Créer mon compte » à l'inscription) non activable sur Android car masqué par la barre de navigation / le clavier. Ce défaut, consigné BUG-011, a été corrigé, puis les scénarios concernés (TF-016, TF-017, TF-023, TF-024) ont été re-testés avec succès — illustrant le cycle complet détection → correction → re-test.

13.2 Tests structurels — résultats exécutés

Test	Cible	Résultat	Statut
Tests unitaires backend	100 % passants	80/80	OK
Couverture services backend	≥ 80 %	94-100 % lignes	OK
Tests e2e API	100 % passants	20/20	OK
Tests unitaires mobile	100 % passants	51/51	OK
Couverture logique mobile	≥ 80 %	100 % (slots/schemas)	OK
Lint backend + mobile	0 erreur	0 erreur	OK
Typage strict (tsc)	0 erreur	0 erreur	OK
Santé Expo (expo-doctor)	0 problème	18/18	OK
Compilation bundle mobile	succès	exporté	OK
Build backend	succès	dist/main.js	OK
Healthcheck API	ok + BDD up	ok	OK
Build EAS preview (APK)	APK produit	produit par le CD	OK

Sécurité — 8/8 OK : les 8 scénarios TS du §9, exécutés réellement contre l'API.

13.3 Automatisation Maestro

Flow	Scénario automatisé
01-inscription-connexion	TF-001 / TF-004
02-reservation-complete	TF-010 / TF-012
03-conflit-creneau	TF-011
04-annulation	TF-015
05-chat	TF-016
06-admin-absent-membre	TF-021

Exécution locale : `maestro test e2e/`.

► *Détail : docs/07-cahier-de-recettes.md, mobile/e2e/.*

14. Plan de correction des bogues — C2.3.2

Processus : détection (outillage local → CI → recette) → qualification via le template d’issue GitHub → correction sur branche `fix/*` avec test de non-régression obligatoire → PR + CI verte → fermeture référencée au commit.

14.1 Grille de qualification

Gravité	Définition	Délai visé
Bloquant	Fonctionnalité inutilisable, perte de données, faille, build/CI cassé	Immédiat
Majeur	Fonctionnalité dégradée sans contournement simple	Avant la prochaine version
Mineur	Gêne ponctuelle avec contournement	Planifié
Cosmétique	Affichage, libellé, alignement	Opportuniste

14.2 Registre des bogues réellement rencontrés (11, tous corrigés — dont 1 issu de la recette)

ID	Gravité	Description	Correction
BUG-001	Bloquant	Typecheck rouge en CI, vert en local (types Expo gitignorés)	@types/node + types:["node"]
BUG-002	Majeur	Prisma 7 incompatible avec l’outillage NestJS 11	Repli sur Prisma 6.19
BUG-003	Majeur	App non lançable (Expo Go plafonné à la SDK 54)	Alignement sur la SDK 54
BUG-004	Majeur	Tests d’interface s’invalidant (RNRTL v14 async)	fireEvent → userEvent
BUG-005	Bloquant	Suite non exécutable (portée des mocks Jest)	Convention mock*
BUG-006	Majeur	Appel impur (Date.now()) pendant le rendu	Hook useNow()
BUG-007	Majeur	Lint bloquant sur les e2e (any de Supertest)	Interfaces de réponse
BUG-008	Mineur	Assertion fausse (mutation TanStack Query)	Assertion ciblée sur le payload

ID	Gravité	Description	Correction
BUG-009	Mineur	Seuil de couverture (gateway chat)	Tests dégradés + seuil ajusté
BUG-010	Bloquant	Vulnérabilité fast-uri bloquant la v1.0.0	npm audit fix → 3.1.4
BUG-011	Majeur	Recette : bouton d'action bas d'écran non activable sur Android (chat + inscription)	Insets système + KeyboardAvoidingView ; re-test 24/24 OK

14.3 Deux fiches détaillées

BUG-001	Typecheck en échec sur la CI alors qu'il passe en local
Cause racine	Le type global process (process.env) était fourni par expo-env.d.ts et .expo/types/, générés par les commandes Expo et volontairement gitignorés. Sur un checkout propre (CI), ces fichiers n'existent pas : la déclaration disparaît. Le typage dépendait d'artefacts non versionnés.
Correction	Ajout de @types/node en devDependency et de "node" au champ types du tsconfig.json ; la déclaration ne dépend plus de fichiers générés.
Non-régression	Reproduction des conditions CI en local (déplacement des fichiers Expo) : échec confirmé puis succès. Le job mobile de la CI, sur checkout propre, est le garde-fou permanent.

BUG-010	Vulnérabilité de dépendance bloquant la publication de la v1.0.0
Cause racine	fast-uri (dépendance transitive de @nestjs/cli, build-time uniquement, absente de l'image de production) : advisory de sécurité haute publié après le dernier passage vert de la CI. Le code du projet n'a pas changé.
Correction	npm audit fix : montée transitive de fast-uri en 3.1.4. Backend ramené à 0 vulnérabilité ; le tag v1.0.0 est reposé sur le commit corrigé.
Non-régression	npm audit --audit-level=high bloquant en CI sur main et les tags : toute vulnérabilité haute réapparaissant fera de nouveau échouer le pipeline. Surveillance continue par Dependabot.

BUG-004	Tests d'interface s'invalidant mutuellement
Cause racine	React Native Testing Library v14 rend de façon asynchrone ; fireEvent déclenche des mises à jour d'état hors de act(), corrompant le root de rendu partagé entre les tests d'un même fichier.
Correction	Remplacement de fireEvent par l'API userEvent (interactions encapsulées dans act()) ; suppression des démontages manuels au profit du nettoyage automatique de RNTL.
Non-régression	Les suites login-screen et room-booking exécutent plusieurs rendus successifs dans un même fichier : toute régression les ferait échouer en CI.

BUG-011	Bouton d'action bas d'écran non activable sur Android (issu de la recette)
Cause racine	Deux causes convergentes : (1) la barre de composition du chat ne réservait aucune marge pour les insets système (barre de navigation gestuelle Android), le bouton d'envoi passant sous la zone non tactile ; (2) les écrans à formulaire utilisaient un KeyboardAvoidingView en behavior="height" sur Android, mode qui cohabite mal avec un ScrollView et masque les éléments du bas clavier ouvert.

BUG-011	Bouton d'action bas d'écran non activable sur Android (issu de la recette)
Correction	Chat : marge basse via useSafeAreaInsets() sur le composeur et behavior={undefined} sur Android. Formulaires (login, register) : KeyboardAvoidingView en behavior={undefined} sur Android (redimensionnement natif + ScrollView). Lint 0/0, typecheck 0, 51/51 tests mobile.
Non-régression	Re-exécution manuelle sur appareil réel des scénarios TF-016, TF-017, TF-023 et TF-024 : les quatre passent désormais (recette fonctionnelle 24/24). Cycle complet détection → correction → re-test conforme.

14.4 Incidents d'environnement (hors code applicatif)

ID	Description	Traitement
ENV-001	Démarrage de l'API impossible : port 3000 déjà utilisé (processus résiduel)	Processus arrêté ; vérification du port documentée dans le manuel de déploiement
ENV-002	npm audit mobile : 15 vulnérabilités moderate (outillage de build Expo)	Sous le seuil bloquant high ; non embarquées dans l'APK ; suivies à chaque montée de SDK

Enseignement : le dispositif a détecté 100 % de ces anomalies avant toute version publiée. BUG-001 et BUG-010 n'étaient détectables que par la CI, ce qui justifie l'environnement d'intégration distinct du poste de travail. BUG-011, seul défaut relevé à la recette, a suivi le même processus — détecté sur appareil réel, corrigé, puis re-testé conforme (recette fonctionnelle 24/24) — démontrant le cycle complet de traitement d'une anomalie de recette.

► Détail : docs/08-plan-correction-bogues.md, .github/ISSUE_TEMPLATE/bug_report.yml.

15. Manuel de déploiement — C2.4.1

15.1 API — prérequis et variables

Prérequis : Docker + Docker Compose (aucune autre installation — tout est conteneurisé).

Variables dans un .env (modèle .env.prod.example), contrôlées au démarrage (\${VAR:?requis}) :

Variable	Description	Obligatoire
POSTGRES_USER / _PASSWORD / _DB	Identifiants PostgreSQL	Oui
JWT_ACCESS_SECRET / _REFRESH_SECRET	Secrets de signature (distincts)	Oui
JWT_ACCESS_TTL / _REFRESH_TTL	Durées de vie des jetons	Non (15m / 7d)
API_VERSION	Tag de l'image à déployer	Non (latest)

15.2 Déploiement depuis zéro

```
git clone <dépôt> && cd SoundProof
cp .env.prod.example .env      # renseigner les secrets
docker compose -f docker-compose.prod.yml up -d
```

```
curl http://localhost:3000/api/v1/health # → {"status":"ok"}
```

Migrations automatiques à l'entrypoint. La BDD n'est pas exposée (réseau Docker internal), l'API l'est sur le port 3000. Rollback : `API_VERSION=<tag précédent> + up -d`.

Sauvegarde (à réaliser avant chaque mise à jour) :

```
docker compose -f docker-compose.prod.yml exec -T db \
  sh -c 'pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" -F c' \
  > sauvegarde-$(date +%Y%m%d-%H%M).dump
```

La restauration s'effectue symétriquement avec `pg_restore --clean --if-exists`. L'encapsulation dans `sh -c` résout les variables dans le conteneur.

15.3 Dépannage (extrait)

Symptôme	Cause probable	Solution
POSTGRES_PASSWORD requis au démarrage	Variable absente du .env	Compléter le .env puis <code>up -d</code>
Le conteneur backend redémarre en boucle	Base pas prête ou DATABASE_URL erronée	logs backend ; vérifier que db est healthy
/api/v1/health renvoie database: down	Base arrêtée ou identifiants invalides	logs db ; vérifier le mot de passe du .env
Port 3000 déjà utilisé	Autre service sur le port	Arrêter le processus ou changer le port publié
L'app mobile ne joint pas l'API	URL/HTTPS/réseau	Vérifier EXPO_PUBLIC_API_URL et le pare-feu

15.4 Application mobile

Build `eas build --platform android --profile preview` → récupération de l'APK (QR code / GitHub Release) → installation (autoriser les sources inconnues). La publication sur Google Play / App Store est documentée mais non exécutée (comptes développeur payants).

► *Détail : docs/09-manuel-deploiement.md.*

16. Manuel d'utilisation — C2.4.1

Destiné aux utilisateurs finaux, il couvre l'installation de l'APK, la création de compte (règle de mot de passe expliquée), la connexion et la session persistante, puis, par profil :

- **Membre** : consulter les salles et leurs disponibilités, réserver un créneau et du matériel, gérer et annuler ses réservations, utiliser le chat de salle.
- **Administrateur** : créer/modifier/désactiver salles et matériel, superviser et annuler toute réservation.

Action administrateur	Comment
Créer une salle	Onglet Admin → « + Nouvelle salle » : nom, description, capacité
Ajouter du matériel	Sur une salle → « Matériel » : nom + catégorie
Désactiver une salle / un matériel	Bouton « Désactiver » + confirmation (désactivation logique, historique conservé)
Annuler une réservation	Onglet Admin → Réservations → « Annuler » sur une réservation future

Chaque écran est décrit pas à pas, avec les règles à connaître (durée 30 min–8 h, anti-chevauchement, accès au chat réservé aux membres de la salle) et une section accessibilité. Les captures d'écran sont référencées dans docs/assets/.

16.1 FAQ (extrait)

Question	Réponse
Que se passe-t-il sans connexion ?	Un message explicite s'affiche (« Connexion impossible... ») ; l'app ne se ferme pas. Actualiser au retour du réseau.
Comment savoir si ma réservation est confirmée ?	Message de confirmation, réservation dans « À venir », créneau grisé dans le calendrier.
Pourquoi mon créneau est-il refusé ?	Quelqu'un a réservé entre-temps, ou un matériel coché est déjà pris. Rafraîchir puis réessayer.
Puis-je modifier une réservation ?	Non directement : annuler puis recréer. L'annulation libère immédiatement le créneau.
Pourquoi n'ai-je pas accès au chat ?	Le chat est réservé aux membres ayant une réservation dans la salle (passée ou à venir).
Jusqu'à quand puis-je annuler ?	Jusqu'au début de la réservation ; une fois le créneau commencé, l'annulation n'est plus possible.
Mes messages sont-ils visibles par tous ?	Uniquement par les membres de la salle et les administrateurs ; chaque salle a sa propre conversation.
Je ne vois pas l'onglet Admin, est-ce normal ?	Oui : il n'apparaît que pour les comptes administrateur ; les comptes créés sont des membres.
Comment mettre à jour l'application ?	Télécharger le nouvel APK depuis la page des versions et l'installer par-dessus ; données et session conservées.

► Détail : docs/10-manuel-utilisation.md.

17. Manuel de mise à jour — C2.4.1

Couvre la montée de version API (sauvegarde → nouveau tag → pull → up -d, migrations automatiques) et mobile (incrément de version dans app.json, build EAS, distribution ; mises à jour OTA eas update mentionnées comme évolution possible).

17.1 Compatibilité API ↔ application

Règle : l'API doit rester compatible avec la version N-1 de l'application mobile (les utilisateurs ne mettent pas à jour immédiatement). Un changement incompatible impose une transition en deux temps, l'API étant toujours déployée avant l'application.

Changement d'API	Compatible ?	Conduite
Ajouter un champ / un endpoint / un paramètre optionnel	Oui	Déploiement direct
Rendre obligatoire un paramètre optionnel	Non	Transition en deux temps
Supprimer / renommer / changer le type d'un champ	Non	Transition en deux temps

- **Migrations Prisma** : création (migrate dev), application automatique (migrate deploy), trois méthodes de retour arrière (restauration de sauvegarde, migration corrective, migrate resolve).
- **Dépendances** : Dependabot + npm audit ; cas particulier Expo (via expo install --fix, jamais Dependabot) ; note sur la montée vers Prisma 7.

17.2 Procédure de correctif urgent (hotfix)

Pour un bogue bloquant en production, sans attendre le cycle normal via develop :

```
git checkout main && git pull
git switch -c fix/description
# corriger + AJOUTER LE TEST DE NON-RÉGRESSION (obligatoire)
npm run lint && npm run typecheck && npm test && npm run test:e2e
# PR vers main, CI verte, fusion, puis :
git tag v1.0.1 && git push origin v1.0.1 # déclenche le déploiement
git checkout develop && git merge main # éviter la régression
```

Une check-list avant toute mise en production (sauvegarde, CHANGELOG, CI verte, recette exécutée, compatibilité API↔app N-1, rollback prêt) clôt le manuel.

► Détail : docs/11-manuel-mise-a-jour.md.

18. Correspondance compétences ↔ preuves

Compétence	Preuve principale (section / dépôt)
C2.1.1 Environnements de déploiement/test + qualité	§2, §3 ; docs/01, docs/02, cd.yml, eas.json, docker-compose*.yml
C2.1.2 Intégration continue	§4 ; docs/03, ci.yml, hooks Husky
C2.2.1 Prototype (ergonomie, équipement, sécurité) △ éliminatoire	§5-7 ; docs/04, mobile/app/, API NestJS
C2.2.2 Harnais de tests unitaires △ éliminatoire	§8 ; 80 + 20 + 51 tests, couverture ≥ 80 % bloquante
C2.2.3 Sécurité + accessibilité △ éliminatoire	§9-10 ; docs/05, docs/06, mesures dans le code

Compétence	Preuve principale (section / dépôt)
C2.2.4 Versions + livraison	§11-12 ; tags SemVer, CHANGELOG.md, Release v1.0.0
C2.3.1 Cahier de recettes  éliminatoire	§13 ; docs/07, mobile/e2e/
C2.3.2 Plan de correction des bogues	§14 ; docs/08, template d'issue
C2.4.1 Documentation d'exploitation	§15-17 ; docs/09, docs/10, docs/11

Les 4 compétences éliminatoires (C2.2.1, C2.2.2, C2.2.3, C2.3.1) sont traitées avec preuves à l'appui, exécutées et vérifiées réellement.